Builder

# Builder Pattern

Construct step by step

DEFINITION

Separate the construction of a complex object from its representation so one process can build different results.

Builder splits construction into a sequence of small steps held by a Builder object, with an optional Director that orders them, so the same process can yield different representations. It targets the telescoping-ctor problem: instead of one call with a dozen positional args, you name each part, validate, then call build() to get a finished, often immutable result.

WHY IT MATTERS

A long positional argument list is unreadable and easy to get wrong — which boolean was "cacheable"? — and partially-built objects invite invalid states. A Builder makes each part explicit and optional, validates before build(), and can return an immutable product, so the half-built state lives in the builder and never in the object you hand out.



Vasyl Krupka · Ukraine

## How it works

| | |
|---|---|
| **Builder** | Holds step methods (one per part) and accumulates the configuration. |
| **Director** | Optional — encodes a recipe that calls the steps in a fixed order. |
| **Product** | The assembled, validated result that build() returns. |

HOW TO APPLY

1. Spot an init with many optional / positional params, or objects that can exist half-built and invalid.
2. Add a Builder whose step methods each set one part and return this for chaining.
3. Validate inside build() and return the finished (ideally immutable) Product.
4. Optionally add a Director to capture a reusable construction recipe.

LITMUS TEST

Are you passing many optional params, or can the object exist half-built and invalid? If naming the parts and validating once at build() would help, use a Builder.

## — Smell → fix

A telescoping ctor — positional args nobody can read at the call site:

```
new Query('users', ['id'], 'age > 18',
null, 10, true, false) // which flag?
// optional params pile up; invalid combos slip through
```

↓ NAME THE PARTS, VALIDATE AT build()

```
const q = new QueryBuilder()
.select('id').from('users')
.where('age > 18').limit(10)
.build() // validates, returns an immutable Query
```

Each step names one part and returns this for chaining; build() validates and emits an immutable Query. No half-built object ever escapes, and the call site reads top to bottom.

USE WHEN

- ✓ Many optional params / telescoping constructors
- ✓ Immutable objects with validation

AVOID WHEN

- ▲ Simple objects with few fields — overkill



Vasyl Krupka · Ukraine

# — Trade-offs & pitfalls

PROS

+ Readable construction
+ Validates before build()

CONS

− More boilerplate
− A two-phase object

COMMON PITFALLS

▲ Reaching for a Builder on a 2-3 field object — it is boilerplate that a plain object literal or named params beat.

▲ Forgetting to validate in build(), so the builder just relocates the same invalid-state problem.

▲ A mutable "product" that keeps its setters after build() — leak the builder, not a half-open object.

WHERE YOU SEE IT

- Query builders (Knex, TypeORM createQueryBuilder) assembling SQL step by step.
- HTTP / request builders and test-data builders (the fluent fixture / "Object Mother").
- StringBuilder and fluent config objects where parts accrue before a final build.

SEE ALSO

| | |
|---|---|
| **Abstract Factory** | returns ready families immediately; Builder assembles one object over several steps |
| **Factory Method** | simple delegated creation; Builder is for multi-step, validated assembly |
| **Prototype** | clone a prepared instance when building from scratch is the costly part |
| **Composite** | Builders are a natural fit for assembling a Composite tree node by node |



Vasyl Krupka · Ukraine

# Interview & sources

### Builder vs Abstract Factory?

Builder constructs one complex object step by step and cares how it is assembled; Abstract Factory returns families of products immediately and cares what they are.

### What problem does Builder actually solve?

Telescoping ctors and invalid half-built state: it names optional parts, validates once at build(), and can hand back an immutable result.

### Is the Director required?

No. The fluent-builder form most TS/JS code uses drops the Director; you add one only to reuse a fixed construction recipe.

### When is Builder overkill?

For small objects with few fields — an object literal or default params is clearer. Builder earns its keep with many optional parts or validation.

REMEMBER

Separate how a complex object is assembled from what it ends up being — build it step by step, then validate and emit.

SOURCES

GoF — "Design Patterns" (1994): Builder (creational)

Joshua Bloch — "Effective Java" (3rd ed., 2018): Item 2 — use a builder when faced with many optional parameters